User manual - Llama Desktop Launcher

Version 0.9.0 · English

This is the first of three guides. The **Integration guide** (INTEGRATION-en) explains how to connect OpenCode, other clients and your own applications to the API. The **Diagnostics guide** (DIAGNOSTICS-en) covers common problems, the app's messages and the support report. All three are linked in the app footer and live in the repository's docs/ folder.

1. What the application does

Llama Desktop Launcher prepares and starts a language model server on your computer and exposes it as an OpenAI-compatible API. It works with three engines:

- **[llama.cpp](https://github.com/ggml-org/llama.cpp)** (tested build b12211) for **GGUF** files. It is the main engine: it has the integrated chat, the terminal CLI, settings suggestions and the benchmark.
- **[vLLM](https://github.com/vllm-project/vllm)** 0.32.0 for **Hugging Face folders** (config.json and .safetensors weights, including AWQ and GPTQ). It runs on Linux through a launcher-managed Python environment and on Windows through Docker Desktop.
- **[Strata](https://github.com/Niko1221/Strata) and [PolyStrata](https://github.com/simplexai-labs/PolyStrata)** for the models those projects install. You install them with their official installer; the launcher only starts the server.

You choose the access: this computer only, local network (LAN) or Tailscale. Outside this computer the API requires a key, and you can share the model with revocable links and commands.

The application installer **does not include models or engines**. The app downloads llama.cpp itself; vLLM and Strata have their own setup (sections 6 and 7).

The interface has three screens, reachable from the top navigation:

- **Llama:** choose the model, start the server, chat and copy client setups.
- **Models:** recommendations for your hardware and links to find models for each engine.
- **Settings:** engines, model folders, profiles, import, updates, diagnostics and, on Windows, the firewall rules created by the launcher.

The launcher version appears in the header, next to “llama.cpp no seu computador”. Each engine has its own version.

2. Requirements

System	Architecture	llama.cpp	vLLM	Strata / PolyStrata
Windows 10/11	x64	Vulkan, NVIDIA CUDA, AMD ROCm, Intel SYCL or CPU	Docker Desktop with WSL2 and an NVIDIA GPU	Yes, through their official installer
Linux (DEB, RPM, AppImage)	x64	Vulkan, CUDA, ROCm, SYCL or CPU	Python 3.10-3.13 and an NVIDIA GPU (CUDA)	Yes, through their official installer
Linux (DEB, RPM, AppImage)	ARM64	Vulkan or CPU	Not tested	Depends on the project
macOS (Apple Silicon and Intel)	ARM64 / x64	Metal	Not offered	Not offered

Disk space depends on the model. Leave room for the weights, the cache and the engine (the llama.cpp CUDA package and the vLLM Docker image are several GB). Large models may need much more memory than their size on disk. Without a compatible GPU, llama.cpp falls back to the CPU and becomes slow; vLLM and Strata require a GPU. On Linux, the official llama.cpp package is built on Ubuntu; other distributions may need compatible libraries.

3. Install the application

Installers and the SHA256SUMS.txt checksum file are on the latest release page. The source code is in the project repository.

System	File in the release (X.Y.Z = version)
Windows x64	Llama.Desktop.Launcher.Setup.X.Y.Z.exe
Debian, Ubuntu, Mint, Pop!_OS	llama-desktop-launcher_X.Y.Z_amd64.deb or _arm64.deb
Fedora, RHEL, openSUSE	llama-desktop-launcher-X.Y.Z.x86_64.rpm or .aarch64.rpm
Any Linux distribution (portable)	Llama.Desktop.Launcher-X.Y.Z.AppImage or -arm64.AppImage
macOS Apple Silicon	Llama.Desktop.Launcher-X.Y.Z-arm64.dmg
macOS Intel	Llama.Desktop.Launcher-X.Y.Z.dmg

Windows

1. Open the .exe installer.
2. Choose the installation folder. The installer is per-user and does not need Program Files.
3. Tick or untick the desktop shortcut. Updates and silent installs (/S) keep your previous choice.
4. Open the app from the Start menu or the shortcut.

Linux

- **DEB / RPM:** install with your distribution's package manager (sudo apt install ./file.deb or sudo dnf install ./file.rpm).
- **AppImage:** make the file executable (chmod +x) and open it. It stays in the folder you choose.

macOS

1. Open the DMG and drag the app to **Applications** (or another folder).
2. Open it from Finder. The packages are not yet signed or notarized, so macOS may ask for permission in **System Settings > Privacy & Security**. Allow only this app; do not turn off system protections.

4. First use

On first launch, the app opens **Settings** while it still lacks a ready engine, a model in the folders or a selected model. Once everything is ready, later launches go straight to **Llama**. You can leave at any time with **Concluir e ir para Llama** (Finish and go to Llama) and come back later.

Suggested route:

1. **Prepare an engine.** Start with llama.cpp (section 5). vLLM and Strata are optional.

2. **Choose the model folders** in **02 · Pastas** (section 8). The first one starts at ~/Models/GGUF; the second is optional.
3. **Get a model.** The **Models** screen shows what fits your hardware and opens the Hugging Face search. Download through the browser into one of the folders and click **Atualizar lista** (Refresh list).
4. **Select the model** on the **Llama** screen and click **Conversar aqui** (Chat here, GGUF only) or **Iniciar servidor** (Start server).

5. llama.cpp engine

In **Settings > 01 · Motor**, choose one option:

1. **Download official engine:** choose the **official package backend** and click the button. The app downloads the ggml-org/llama.cpp b12211 release from GitHub for your platform and checks the SHA-256 when the release provides the digest. The engine is stored under engines/ in the app data folder, outside the installation.
2. **Choose existing folder:** pick a folder that already contains llama-server (or llama-server.exe), such as the b11193-vulkan folder of the previous Windows launcher.

Choose the backend by GPU, not by CPU brand:

- **Vulkan:** works with AMD, Intel and NVIDIA GPUs with up-to-date drivers. It is the most universal option.
- **NVIDIA CUDA:** best performance on NVIDIA cards. The download includes the CUDA libraries; you only need a compatible driver. For an RTX 4060 on Windows, start with **NVIDIA CUDA 12.4**.
- **AMD ROCm / Intel SYCL:** x64 only, and they may need extra runtimes.
- **CPU:** no acceleration. **Metal:** macOS.

The download is extracted into a temporary folder next to its final destination; if it fails, the folder is removed and the previous engine stays configured. To change engines, download another backend or choose another folder; stop the server and the chat first.

Import profiles from the previous launcher

If you used the WinForms launcher, set up the folder with your GGUF files first. Then, in **Settings > 04 · Importação**, click **Importar perfis TSV** and choose launcher-profiles.tsv. Profiles are matched by the model's full path; profiles already saved are not overwritten. The result shows how many were imported, not found, already present or invalid.

6. vLLM engine

vLLM serves models in Hugging Face format: a folder with config.json and .safetensors weights (including AWQ and GPTQ). Put these folders inside the model folders; they appear in the list with the · vLLM suffix. Hugging Face cache folders (models--org--name/snapshots/<hash>) are also recognized.

Linux

1. Install python3 and python3-venv (Python 3.10 to 3.13) and the NVIDIA driver with CUDA.
2. In **Settings > 01B · Outros motores**, click **Instalar vLLM**. The launcher creates a Python environment in the data folder and runs pip install vllm==0.32.0 (several GB). Progress appears in the **Registro do motor** (engine log) on the **Llama** screen.
3. If you already have an environment with vLLM, use **Usar ambiente existente** and choose the folder that contains bin/vllm.

The pip package only supports NVIDIA CUDA. For AMD (ROCm), install vLLM following the official documentation and point to that environment.

Windows

1. Install Docker Desktop with the WSL2 backend and NVIDIA GPU support, and keep it running.
2. In **01B · Outros motores**, click **Baixar imagem do vLLM**. The launcher pulls vllm/vllm-openai:v0.32.0 (about 10 GB). Use **Verificar de novo** (Check again) after opening Docker Desktop.
3. When the server starts, the model is mounted in the container **read-only** and without internet access (HF_HUB_OFFLINE=1): the folder must be complete. The container stops with the server.

What vLLM uses

- The vLLM profile has its own fields: GPU memory utilization, KV cache type, tensor parallelism, speculative decoding and more (section 10). **Simultaneous sessions** become --max-num-seqs.
- The first load can take several minutes (graph compilation). The launcher waits up to 30 minutes.
- **Abrir no navegador** (Open in browser) opens the API documentation (/docs), because vLLM has no chat interface.
- vLLM is not offered on macOS.

7. Strata and PolyStrata

Strata and PolyStrata serve specific model families with their own quantization and GPU usage. They work on Windows and Linux.

1. Install with the project's official installer (Strata or PolyStrata), using START-HERE.bat (Windows) or setup.sh (Linux), and configure at least one model. Their installer downloads the weights and creates the .venv folder.
2. In **01B · Outros motores**, click **Escolher pasta instalada** and select the folder that contains serve/server.py and .venv.
3. Each strata-<model>.json file in that folder becomes a list item, with the · Strata or · PolyStrata suffix.

The launcher does not download or run these projects' installers. Context, quantization and GPU come from the strata-*.json configuration; the launcher sets the address, port and API key. Extra server options (GPU, queue, unload when idle, minimum VRAM) appear in the profile when the installed version accepts them (section 10). The launcher's **Simultaneous sessions** apply to Strata only; PolyStrata uses the value from its own configuration.

To discover models, use the **Outros motores** card on the **Models** screen. It links to the Strata, PolyStrata and vLLM model lists and says whether the detected GPU meets each one's requirements.

8. Folders and models

The app accepts **two model folders** in **Settings > 02 · Pastas**. Use **Alterar** (Change) to pick another disk, **Abrir** (Open) to view the folder and **Remover** (Remove) to clear the second one. The scan includes subfolders.

Engine	What counts as a model	How it appears in the list
llama.cpp	.gguf files; for split models, only the first shard	File name
vLLM	Folders with config.json and .safetensors	Folder name · vLLM
Strata / PolyStrata	strata-*.json in the installed Strata folder	model_name · Strata or · PolyStrata

For GGUF:

- On the **Models** screen, use **Explorar todos os GGUF**, **Buscar quantizados**, **Buscar MoE** or **Buscar densos F16** to open Hugging Face in the browser. Q4_K_M is a good starting point for the balance between memory and quality.
- Check the license, memory requirements and the author's instructions before downloading.
- If the model uses a multimodal projector, keep a single mmproj*.gguf in the same folder; it is attached automatically.
- Keep all shards of a split model together.

GGUF describes the format, but does not guarantee compatibility with b12211; some models need a newer engine. The app does not accept model folders inside the installation or inside its internal data folder.

The **Models** screen estimates use RAM, CPU, detectable GPU/VRAM and free space. They are conservative starting points, not guarantees. For MoE models, consider the total weight size, not just the active parameters.

9. Llama screen: chat and start the server

1. Choose the model in the **Modelo** list. The line below shows its size, path and engine.
2. Choose the **Endereço de acesso** (access address) and **Sessões simultâneas** (simultaneous sessions, 1 to 8) under **Acesso e sessões**.
3. Use one of the buttons:

- **Conversar aqui** (GGUF only): opens llama-cli inside the window. Enter sends; Shift+Enter adds a line. History is not saved.
- **Abrir CLI no terminal** (GGUF only): opens llama-cli in a separate terminal with the current model and settings. On Linux, a graphical terminal must be installed.
- **Iniciar servidor:** loads the model and shows the API endpoint without opening the browser.
- **Abrir no navegador:** starts the server if needed and opens the engine's interface. With the server running, it only opens the page.
- **Parar servidor:** stops the server, sharing and, for vLLM on Windows, the container.

Only one run at a time: stop the server, chat or benchmark before starting another. Closing the browser tab does not stop the server; closing the launcher does.

The **Registro do motor** (engine log), below the buttons, shows the engine output and the launcher's messages. It is the first place to look when something fails (see the Diagnostics guide).

What each engine offers

Feature	llama.cpp	vLLM	Strata / PolyStrata
Integrated chat and CLI	Yes	No	No
Initial suggestion	Yes	Yes	No
Autotuning	llama-bench and a short llama-server run	Optional; reloads the model on each attempt	No
Open in browser	llama-server web interface	API documentation (/docs)	Strata server page
Profile settings	llama.cpp's	vLLM's	Sessions (Strata only) and options the installed server.py accepts
Maximum load wait	5 minutes	30 minutes	30 minutes
Model ID in the API	modelo-local	modelo-local	model_name from the config

10. Per-model profiles

Each model has its own profile, saved by its file or folder path. The profile is saved when you click **Salvar ajustes deste modelo** (Save this model's settings) and also when you start a server or chat with that model.

On the **Llama** screen, the settings are in the **Ajustes avançados deste modelo** accordion, closed at first. In **Settings > 03 · Perfil do modelo**, the same fields are shown open, with a model selector.

How the panel works

- **Only the chosen model's engine parameters are shown.** A GGUF shows llama.cpp's; a Hugging Face folder, vLLM's; a strata-*.json configuration, Strata's.
- **What the model or the hardware does not support is collapsed** under **Não se aplicam a este modelo (N)** (Not applicable to this model), each with its reason. Examples: "Modelo denso: não tem especialistas MoE" (dense model, no MoE experts), "Só com 2 ou mais GPUs" (needs 2 or more GPUs), "O modelo não usa atenção com janela deslizante (SWA)".
- **Each field has an Auto box.** When checked, the launcher does not pass the option and the engine decides. Below the field, the automatic value is shown when it can be known, with its source: Automático: 25 · ajuste automático (fit).
- The chips at the top summarize what the launcher read from the model: architecture, layers, training context and type (dense, MoE, SWA, hybrid/recurrent, MTP).

The launcher finds out what the model supports without loading the weights: it reads the GGUF header (experts, MTP/NextN layers, sliding window, recurrent layers), the Hugging Face folder's config.json (experts, MTP, quantization, maximum context, attention heads) and the GPUs (llama-server --list-devices for llama.cpp, nvidia-smi for vLLM and Strata).

llama.cpp (GGUF)

Group	Field	Argument	Shown when
GPU and memory	GPU layers	-ngl	There is a GPU. Value: a number, or all.
	Context	-c	Always. On Auto, -fit picks the largest that fits.
	Minimum context for automatic fitting	-fitc	Context on Auto.
	Free margin per GPU (MiB)	-fitt	There is a GPU.
	Device	-dev	There is a GPU. Names from --list-devices (e.g. CUDA0, Vulkan1); several separated by commas; none = CPU only.
	Split across GPUs / Share per GPU	-sm / -ts	2 or more GPUs.
	KV cache off the GPU	-nkvo	There is a GPU.
	Load mode	-lm	Always: auto, none, mmap, mlock, mmap+mlock, dio.
MoE experts	Expert layers on CPU	-ncmoe	MoE model.
	Expert cache on GPU (MiB)	--moe-cache-mib	MoE model with a GPU.
	Share of cache misses computed on CPU	--moe-cache-cpu-share	Expert cache above 0.
Dense model	Layers with FFN on CPU	-ncffn	Model without experts, with a GPU.
CPU and batches	Generation / prompt threads	-t / -tb	Always.
	Batch / Microbatch	-b / -ub	Always. The microbatch cannot exceed the batch.
KV cache and attention	K / V cache	-ctk / -ctv	Always. A quantized V cache requires Flash Attention.
	Flash Attention	-fa	Always.
	Full SWA cache	--swa-full	Model with a sliding window.
	Context checkpoints per session	-ctxcp	SWA, hybrid or recurrent model.
	Cache reuse by shifting	--cache-reuse	Server only.
	Unified KV cache across sessions	-kvu	Server with 2 or more simultaneous sessions.

Group	Field	Argument	Shown when
Speculative decoding	Type	--spec-type	Always: none, ngram-mod, ngram-simple, ngram-map-k; draft-mtp only if the GGUF has MTP layers; draft-simple/draft-eagle3 with a draft model.
	Draft model	-md	Types draft-simple and draft-eagle3. Lists the smaller GGUFs in your folders.
	Tokens per draft	--spec-draft-n-max	draft-* types.

The launcher always passes -fit on: with fields on Auto, llama.cpp fits layers, context and experts on CPU to the free memory. **Simultaneous sessions** (--parallel) split the context: with context 8192 and 2 sessions, each client gets 4096 tokens.

vLLM (Hugging Face folder)

Field	Argument	Shown when
Context	--max-model-len	Always. On Auto, the model's max_position_embeddings applies.
GPU memory utilization	--gpu-memory-utilization	Always (0.05 to 0.99; default 0.9).
KV cache type	--kv-cache-dtype	Always: auto, float16, bfloat16, fp8, fp8_e4m3, fp8_e5m2. fp8 depends on the GPU.
Weights in RAM / KV cache in RAM (GiB)	--cpu-offload-gb / --kv-offloading-size	Always.
Tokens per batch	--max-num-batched-tokens	Always.
Prefix cache	--enable-prefix-caching / --no-enable-prefix-caching	Always.
No CUDA graphs	--enforce-eager	Always.
Precision	--dtype	Only for a non-quantized model.
GPUs (tensor parallelism)	--tensor-parallel-size	2 or more NVIDIA GPUs; the number must divide the attention heads.
Expert parallelism	--enable-expert-parallel	MoE model with tensor parallelism above 1.
Speculative decoding	--speculative-config	ngram always; mtp when config.json declares MTP layers. The launcher builds the JSON.

Strata and PolyStrata

Context, quantization and GPU come from the strata-*.json configuration and are shown read-only. The editable fields (**GPU(s)** --gpu, **Max queue** --max-queue, **Unload when idle** --idle-unload and **Minimum free VRAM** --min-free-vram-mib) are enabled only if the installed serve/server.py accepts the option; the launcher checks the file itself, because these projects change quickly.

Suggestion, automatic values and autotuning

- **Aplicar sugestão inicial** (Apply initial suggestion) fills the fields from RAM, CPU, GPU/VRAM and the model data. For a GGUF, it turns on draft-mtp when the model has MTP layers. For vLLM, it computes context, memory utilization and tensor parallelism from config.json and the VRAM, then asks whether you want to run autotuning. Not available for Strata.
- **Calcular valores automáticos** (Compute automatic values, GGUF only) runs llama-fit-params, which ships with the official package, and shows in the Auto fields what -fit would choose: GPU layers, context, share per GPU and how many expert layers would stay on the CPU. It runs by itself when you open a GGUF profile and takes a few seconds.
- **Otimizar automaticamente** (Optimize automatically) searches for the fastest values within the chosen **Tempo** (time: 5, 10, 20 or 30 minutes) and **Prioridade** (priority): **Geração** (generation, the default; generation tokens/s), **Prompt longo** (long prompt; prompt tokens/s) or **Equilíbrio** (balance). **Cancelar** stops the test and applies the best so far. The result goes into the fields, but the profile is saved only when you click **Save**. **Medições do autotuning** lists every attempt, including failures and their reasons.
- **llama.cpp:** first llama-fit-params sets the memory ceiling. Then llama-bench tests, one at a time, experts on CPU (MoE), Flash Attention, K/V cache (f16, q8_0, q4_0), microbatch, threads (when the CPU does work) and GPU split. If time is left, a short llama-server run measures what llama-bench cannot: speculative decoding (with the draft acceptance rate), prompt threads and the expert cache on the GPU. What does not fit in memory or is not supported (a quantized V cache without Flash Attention, for example) is dropped. A value replaces the current one only with a gain of at least 2%.
- **vLLM:** each attempt starts the server again (on Windows, a container) and reloads the model, so each attempt takes several minutes: prefer 20 or 30 minutes. It tests GPU memory utilization, KV cache type, tokens per batch and speculative decoding.
- **Strata/PolyStrata:** there is no autotuning.

Once the server is up, the **Em uso** (In use) line below the endpoint shows the real values: context, GPU layers and sessions for llama.cpp (read from /props and the log); KV cache size and maximum concurrency for vLLM.

Profiles saved before version 0.9.0

Old profiles keep working with the same values. The first time 0.9.0 opens, they get version: 2; -ngl auto and -fa auto now show as Auto, which has the same effect. New fields start on Auto.

Analysis by the model

On the **Models** screen, **Copiar prompt** copies a request to analyze the hardware and settings; **Analisar com modelo selecionado** sends it through the integrated chat. The answer is only a recommendation and changes nothing by itself.

11. Network access and sharing

Under **Acesso e sessões**, the **Endereço de acesso** list shows:

- **Somente este computador (127.0.0.1):** no key; only programs on this machine can connect.
- **Rede interna** (local network): private addresses (10.x, 172.16-31.x, 192.168.x). Requires the API key.

- **Tailscale:** 100.64.0.0/10 addresses on your tailnet. Requires the key and is the preferred option, because traffic is encrypted.

On the network, use **Mostrar chave** (Show key), **Copiar chave** (Copy key) or **Nova chave** (New key; stop the server first). With the server running on LAN or Tailscale, the **Conectar aplicativo** (Connect application) panel offers:

- **Copiar link com acesso:** opens the interface in another computer's browser without typing the key.
- **Copiar Bash/PowerShell com acesso** and **Copiar configuração com acesso:** carry a temporary credential.
- **Revogar acesso compartilhado:** invalidates everything shared and drops open connections. Stopping the server or closing the launcher also ends sharing.

Whoever receives a link or command with access can use the model, but cannot reach the files on your computer. On Windows, the **Firewall do Windows** section creates the inbound rule for the active port. The rule stays after the server stops; remove it in **Settings > 07 · Firewall do Windows**. Ports, keys, clients and firewall details are in the **Integration guide**.

12. Data, updates and uninstalling

Preferences and profiles are stored in settings.json, in the app data folder. The same folder holds the API key (server-api-key.txt), the downloaded engines (engines/) and update packages (updates/). The Diagnostics guide lists the path on each system. Models stay in the folders you chose.

Update the application

At startup and once a day, the app checks the latest release on GitHub. When a new version exists, the header shows **vX.Y.Z disponível**. Open **Settings > 05 · Atualizações** to read the notes. Untick **Verificar atualizações ao iniciar** to turn the check off and use **Verificar agora** (Check now) whenever you want.

Baixar e instalar (or **Baixar atualização**) downloads your system's package and checks it against the release's SHA256SUMS.txt before using it.

- **Windows:** the app asks to stop the server and chat, opens the installer and quits.
- **macOS:** the disk image opens; drag the app to **Applications** and replace the current version.
- **Linux AppImage:** the new AppImage is saved next to the current one, already executable.
- **Linux DEB/RPM:** the verified package is shown in its folder, with the sudo apt install or sudo dnf install command. Close the app before installing.

Engines, models, profiles and the API key are kept. The installers are not yet code-signed: the check protects against corrupted downloads, not against a tampered release.

Uninstall

Stop the server and close the app first. On Windows, use **Settings > Installed apps**; the uninstaller asks you to close the app if it is still open and **keeps** models, profiles and engines. On Linux, remove the DEB/RPM with the package manager or delete the AppImage. On macOS, move the app to the Trash. On every system, user data and models stay for you to review and delete manually if you want. vLLM on Windows leaves the vllm/vllm-openai image in Docker Desktop (remove it there), and the Strata/PolyStrata installation stays in its own folder.

13. Credits

llama.cpp was started by Georgi Gerganov and is maintained by the ggml-org contributors under the MIT license. vLLM belongs to the vllm-project, under the Apache 2.0 license. Strata was created by Niko1221 and contributors under the MIT license. PolyStrata was created by VecSzn and contributors, based on Strata, under the MIT license.

This independent launcher was developed for Célio with Codex, based on the previous WinForms launcher, and is released under the **MIT license**, copyright Célio. It does not redistribute these engines: it downloads the official llama.cpp package, installs the official vLLM package or image, and only starts Strata and PolyStrata installations made by their official installers. The authors of those projects do not participate in or endorse this application. Each model has its own license.